



**BAYAN | بيان**

## **Saudi Regulations & Decisions Data Hub**

Automated Regulatory Data Collection, Version Tracking & Change Detection

### **Final Data Engineering Capstone Report**

#### **Prepared by**

Norah Aldakhil

Yaqeen Alhalal

Shouq Alotaibi

#### **Prepared for**

Saudi Digital Academy - Data Engineering Capstone Project

In collaboration with WeCloudData

Group 01

September 2026

## Abstract

BAYAN is a data engineering platform developed to collect, preserve, structure, version, and analyze Saudi regulatory information. The current implementation focuses on regulations published by the Bureau of Experts at the Council of Ministers (BOE). Instead of treating regulations only as web pages or documents, BAYAN converts them into structured, traceable data that can support historical monitoring and analytical use. The implemented pipeline preserves raw source artifacts in SeaweedFS, transforms regulatory content into a normalized PostgreSQL Silver layer, prepares analytics-ready Gold structures, orchestrates the workflow with Apache Airflow, and presents results through Power BI. The project places particular emphasis on version history, change detection, data quality, and repeatable pipeline execution.

## 1. Introduction

Saudi regulations and decisions are published through official digital channels and may appear as HTML pages, text content, or downloadable documents. For legal, compliance, governance, research, and business users, finding the current text is only one part of the problem. A reliable data foundation should also preserve earlier versions, identify whether a regulation or article has changed, retain important dates and status information, and make the content suitable for structured querying and analysis.

### 1.1 Problem Statement

The original proposal identified four connected challenges: regulatory information can be distributed across official sources; documents may appear in different formats; historical versions are difficult to follow when pages are replaced or updated; and manual comparison requires significant time and attention. The final implementation therefore treats regulatory monitoring as a data engineering problem rather than only a scraping task. The central question is not simply where a regulation can be found, but how its source, structure, versions, dates, articles, and detected changes can remain connected over time.

### 1.2 Project Aim

The aim of BAYAN is to establish a clear and reliable data pipeline for Saudi regulatory content. The system is intended to preserve official source artifacts, standardize regulatory data, maintain version history, detect changes, automate recurring processing, and make curated results available for analysis. The architecture is also designed so that additional official sources and richer regulatory-intelligence features can be added in later versions.

## 1.3 Project Objectives

The proposal defined a set of objectives that guided the implementation. During development, some objectives were delivered directly while others were refined into future work as the team prioritized a reliable end-to-end data pipeline.

- Collect regulatory documents from an official Saudi source and preserve the original source artifacts.
- Keep historical regulation versions instead of replacing earlier records.
- Detect changes through normalized content fingerprints and version-aware processing.
- Improve data quality through cleaning, standardization, relational constraints, and validation checks.
- Structure regulations, chapters, articles, dates, statuses, and versions for SQL-based analysis.
- Provide an analytical user-facing output through Power BI.
- Design the pipeline so additional official sources and more advanced comparison features can be added later.

## 1.4 Target Users

The target audience described in the proposal and final presentation includes law students, lawyers and legal researchers, compliance and governance teams, government employees, businesses and analysts, and researchers or academics. Their common need is not only access to legal text, but clearer organization, historical traceability, and reduced manual effort when monitoring changes. A compliance professional, for example, may need to know whether a provision changed and when the new version became effective, while a researcher may need structured data for analysis across many regulations.

## 1.5 Project Scope and Scope Evolution

The initial proposal scoped one official source, data cleaning and quality, version history, hashing-based change detection, daily automation, and a search/comparison experience. As implementation progressed, the final MVP prioritized the data foundation itself: BOE extraction, SeaweedFS Bronze storage, PostgreSQL Silver and Gold layers, Airflow orchestration, version tracking, validation, and Power BI reporting. Detailed line-by-line text comparison and a dedicated public web portal were moved to future work. This change reduced interface scope while strengthening the completeness and reliability of the core data pipeline.

## 2. Market Context and BAYAN Positioning

Saudi legal information is already served by official and commercial digital platforms. The BOE provides an official source for Saudi regulations and decisions. Qanoniah positions itself as a legal search platform with more than 100,000 Saudi legal documents and describes amendment history and update notifications among its features [2]. Tashree publishes Saudi legal and regulatory documents with metadata such as document type, issuing authority, date, status, and an official-source reference [3]. Other legal-technology products also address AI-assisted legal research and drafting. These products demonstrate that legal discovery, research, and regulatory monitoring are established digital use cases.

BAYAN is not presented as a replacement for these platforms. Its emphasis is the data-engineering layer underneath regulatory intelligence. The project preserves raw source artifacts, models regulations and article versions as structured entities, maintains version history, records detected changes, and produces analytics-ready tables. This makes the current system complementary to reader-facing legal research products: BAYAN focuses on making regulatory data structured, traceable, and reusable for downstream analytics or future applications.

### 2.1 Proposed Solution

The solution can be summarized as four connected stages: collect official regulatory content, process it through a layered data pipeline, track versions and changes, and analyze the curated results. In the proposal, this idea was described as collecting official documents, cleaning and validating data, storing version history, detecting changes automatically, and enabling search and comparison. The final system implements the collection, layered storage, version-aware processing, change tracking, orchestration, and Power BI analytics portions of that concept.

### 2.2 User Value

- Find: regulatory content is organized into structured entities instead of remaining only as separate pages or files.
- Track: version numbers and historical records make the current and previous state more visible.
- Understand: Power BI summarizes status, publication trends, article counts, and change-monitoring indicators.
- Decide: structured, traceable data can reduce the manual effort needed before legal or compliance analysis.

*BAYAN's differentiation is architectural: it adds a raw-to-analytics regulatory data pipeline and version-aware model rather than claiming to replace existing legal research platforms.*

### 3. Data Source and Collection

The current MVP uses the Bureau of Experts at the Council of Ministers as its primary official source. The project targets the BOE regulations portal (laws.boe.gov.sa), which provides structured regulatory pages and, for some regulations, official translated documents. The proposal also identified the Ministry of Justice, Ministry of Commerce, and Umm Al-Qura Gazette as possible future sources; these are not part of the current implementation.

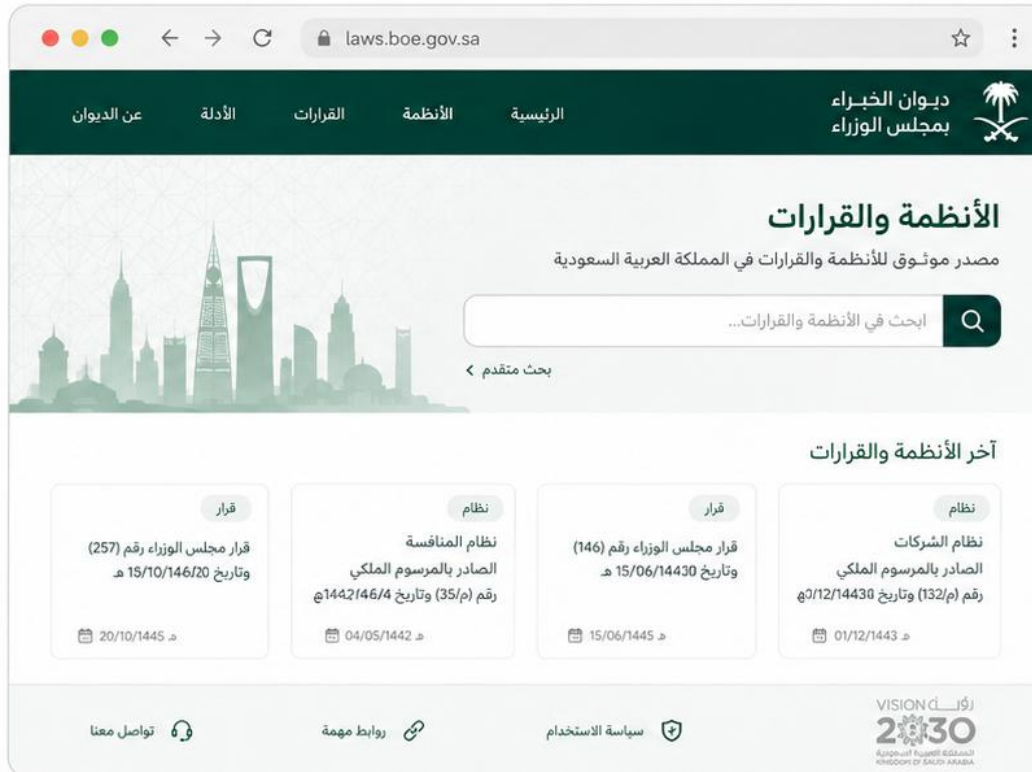


Figure 1: Bureau of Experts regulations portal used as the primary source in the BAYAN MVP.

#### 3.1 Extraction

Python is used to retrieve BOE content, parse regulatory pages, extract metadata and regulatory text, and identify an official translated document when one is available. The extraction code includes handling for network, SSL, timeout, and optional proxy behavior. Each collection run uses a collection date, allowing raw source objects and downstream processing to remain associated with the same run partition.

#### 3.2 Raw Artifacts

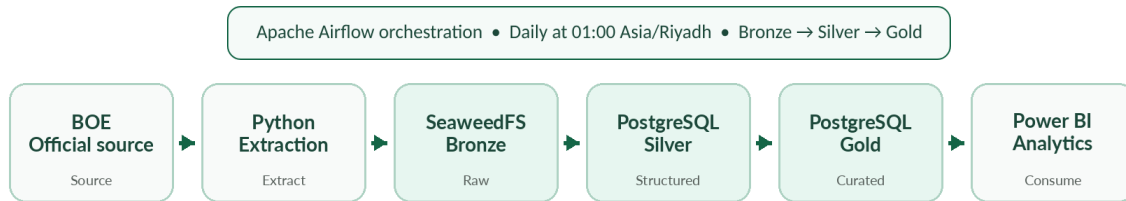
The submitted Bronze snapshot contains 538 regulation folders with a raw.txt artifact and 160 translated.pdf files where an official translated document was available. Raw objects are organized by collection date and regulation identifier. Keeping the raw copy before transformation provides traceability, supports reprocessing when parsing logic changes, and gives the team a source artifact that can be inspected when downstream results require validation.

## 4. System Overview

The final architecture follows an ELT-style pattern: source content is extracted and loaded into raw object storage before it is transformed into structured relational and analytical layers. This separation allows source fidelity and downstream modeling to evolve independently. Apache Airflow coordinates the stages and Power BI consumes the curated output.

### BAYAN end-to-end data flow

Implemented path from official source to analytics



Architecture verified from the submitted pipeline code. Power BI consumes the curated Gold layer.

Figure 2: Implemented BAYAN end-to-end architecture from BOE source to Power BI.

### 4.1 Main Workflow

- Python collects regulation pages, metadata, and available official translated documents from the BOE source.
- SeaweedFS preserves the raw objects as the Bronze layer.
- Python transforms the collected content and loads normalized regulatory entities into PostgreSQL Silver.
- SQL refresh logic creates curated PostgreSQL Gold structures for analytics and change monitoring.
- Apache Airflow runs the Bronze → Silver → Gold dependency chain every day at 1:00 AM Asia/Riyadh.
- Power BI reads the Gold layer and provides the current analytical interface.

### 4.2 Implemented Components

The final implementation includes Python extraction and processing, SeaweedFS raw object storage, PostgreSQL Silver and Gold schemas, SQL validation and Gold-refresh scripts, Apache Airflow orchestration, Docker-based supporting services, DBeaver for database inspection and development, and Power BI for analytics. A separate frontend portal is not part of the current release.

## 5. Layered Data Architecture

BAYAN separates the data lifecycle into Bronze, Silver, and Gold layers. Each layer has a distinct responsibility: Bronze preserves the source, Silver provides structured consistency, and Gold prepares the data for reporting and analytical consumption.

### Layered data architecture

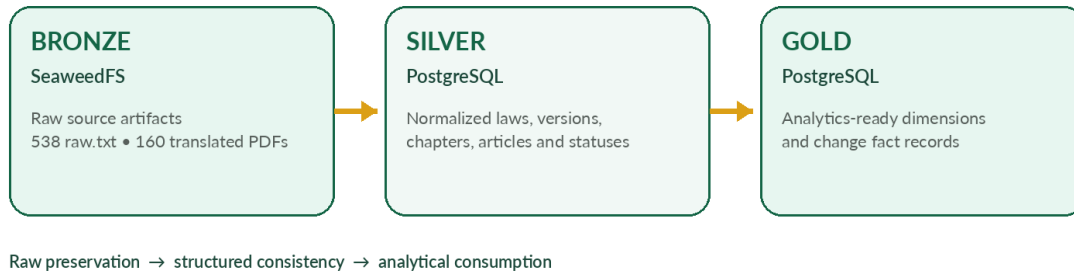


Figure 3: Bronze, Silver, and Gold layers in the implemented data architecture.

### 5.1 Bronze - Raw Preservation

Bronze uses SeaweedFS with an S3-compatible interface. Raw artifacts are kept outside the relational model so the original collected content is available for audit, debugging, and reprocessing. This also prevents the initial collection stage from imposing a rigid relational schema before the content has been interpreted.

### 5.2 Silver - Clean and Structured Data

Silver uses PostgreSQL to model the regulatory domain in normalized form. Verified implementation structures include sources, status, laws, laws\_versions, chapters, articles, articles\_versions, law\_version\_extraction\_status, and processed\_bronze\_objects. Transformation logic normalizes Arabic regulatory headings, parses dates and effective-date rules, creates content hashes, and uses existence or uniqueness checks to reduce duplicate processing.

### 5.3 Gold - Analytics-Ready Data

Gold uses PostgreSQL to simplify analytical consumption. The implemented tables are dim\_regulations, dim\_articles, dim\_date, dim\_change\_type, and fact\_article\_changes. Gold is refreshed from Silver through SQL. The current fact-loading logic records detected Updated article events; Added and Deleted exist as defined change types but are not populated by the current fact-loading query.

## 6. Data Model, Versioning and Change Detection

Historical traceability is a central requirement of BAYAN. The structured model separates a regulation from its versions and connects chapters, article content, and article versions to the appropriate regulatory history. The main relationship can be summarized as Source → Regulation → Regulation Version → Chapter → Article → Article Version. Status and date structures support additional regulatory context, while Gold provides a reporting-oriented view.

### Version tracking and change detection



Article versions are linked to regulation versions; is\_changed is set by comparing stored article hashes.

Figure 4: Version tracking and change-detection logic used by BAYAN.

### 6.1 Regulation Versioning

Before hashing, transformation logic normalizes non-semantic text differences such as whitespace. A SHA-256 content fingerprint is then generated. When a collected regulation matches an existing version, the existing version can be reused; when the normalized content differs, a new version is stored with an incremented version number. This design avoids creating new historical versions for formatting-only differences.

### 6.2 Article Versioning

Article versions are associated with regulation versions. Logical article matching uses the law, chapter title, and article title context so an updated article can be traced even when a new physical article row is created for changed content. Stored hashes are compared to set the is\_changed indicator. This gives the project structured evidence that a particular article changed between collected versions.

### 6.3 Planned Detailed Comparison

The proposal originally included a line-by-line comparison experience. The current implementation provides the version-aware data foundation and change indicators, but not a complete visual diff of added, removed, and modified text. Detailed side-by-side comparison is therefore retained as future work rather than presented as a completed feature.

## 7. Orchestration, Infrastructure and Technology Selection

### 7.1 Apache Airflow

The submitted boe\_pipeline DAG uses the Asia/Riyadh timezone and the cron schedule `0 1 * * *`, corresponding to a daily 1:00 AM run. catchup is disabled and max\_active\_runs is set to one. The same collection date is passed to Bronze and Silver so both stages operate on the same SeaweedFS partition, and the dependency chain is explicitly Bronze → Silver → Gold. Airflow was appropriate because the project required a repeatable multi-stage workflow with explicit dependencies rather than independent manual scripts [4].

### 7.2 Docker and Supporting Environment

Docker and Docker Compose provide a reproducible multi-service environment for the database, object storage, orchestration, and pipeline components. Containerization reduces environment drift, isolates dependencies, and simplifies service networking and repeated setup. The trade-off is additional container and network configuration, which became part of the engineering work during integration [7].

### 7.3 Why These Technologies Were Selected

- SeaweedFS: object-storage semantics and an S3-compatible interface made it suitable for raw Bronze artifacts while remaining self-hosted during the capstone [5].
- PostgreSQL: relational keys, constraints, joins, and SQL querying fit the strong relationships among regulations, versions, chapters, articles, statuses, and dates [6].
- Python: supported HTTP extraction, HTML parsing, text transformation, database interaction, and object-storage integration in one implementation language.
- SQL: used for querying, Gold-layer refresh logic, constraints, and validation checks.
- DBeaver: used as a development and inspection tool for PostgreSQL; it is not a pipeline layer.
- Power BI: used as the analytical consumption layer for the curated Gold data [8].

## 8. Data Quality, Reliability and Engineering Challenges

### 8.1 Data Quality and Validation

Data quality is handled at several points rather than as a single final check. Silver transformation standardizes fields, titles, dates, and version fingerprints; the load layer uses existence checks and relational constraints; processed Bronze object keys support safer reruns; and Gold validation SQL checks analytical consistency. The supplied `validate_gold.sql` includes checks for duplicate regulation and article keys, orphan articles, missing current versions, invalid fact dates, orphan fact references, duplicate change events, and consistency between changed Silver article versions and Gold Updated facts.

### 8.2 Engineering Challenges

The proposal anticipated large document volumes, differences between old and new versions, and changing website structures. During implementation, the team also encountered practical issues that shaped the final design:

- Different source formats and page structures required flexible extraction and parsing logic.
- Network connectivity, SSL behavior, and proxy differences varied across environments.
- Arabic headings and regulatory text required normalization before stable comparison and hashing.
- Database constraints and text-length issues required schema and loading adjustments.
- Duplicate prevention and idempotent reruns required processed-object tracking and ON CONFLICT logic.
- Airflow DAG parsing, Docker networking, SeaweedFS object handling, and cross-layer orchestration required repeated integration testing.
- Change-detection logic had to distinguish true content changes from non-semantic text differences.

### 8.3 What Worked Well

The most successful engineering outcomes were the separation of raw and structured storage, a version-aware relational model, explicit Bronze → Silver → Gold orchestration, repeatable SQL refresh logic, targeted validation, and a Power BI output connected to curated data. Together, these components demonstrate an end-to-end data engineering solution rather than an isolated collection script.

## 9. Power BI Dashboard and Results

Power BI is the current user-facing analytical output of BAYAN. The dashboard consumes curated Gold data and provides a high-level view of regulation status, publication history, article counts, regulation detail, and change-monitoring indicators.

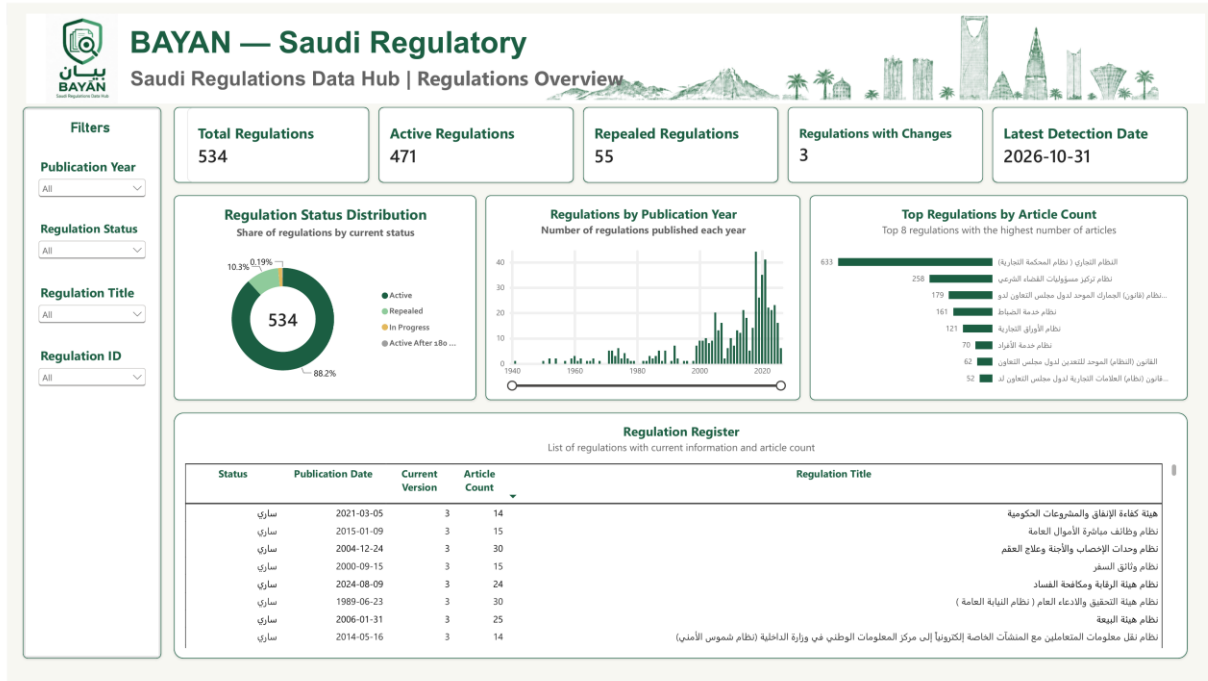


Figure 5: Final BAYAN Regulations Overview dashboard.

### 9.1 Final Dashboard Indicators

The final Regulations Overview dashboard shows 534 total regulations, including 471 active and 55 repealed regulations. It also presents publication-year trends, the regulations with the highest article counts, filters, and a regulation register. The project package contains 538 raw regulation objects in Bronze, showing that the raw collection layer is broader than the final dashboard representation; the submitted evidence does not provide a final run log that explains every excluded raw object, so the report does not infer a cause.

## 9.2 Article and Change Monitoring

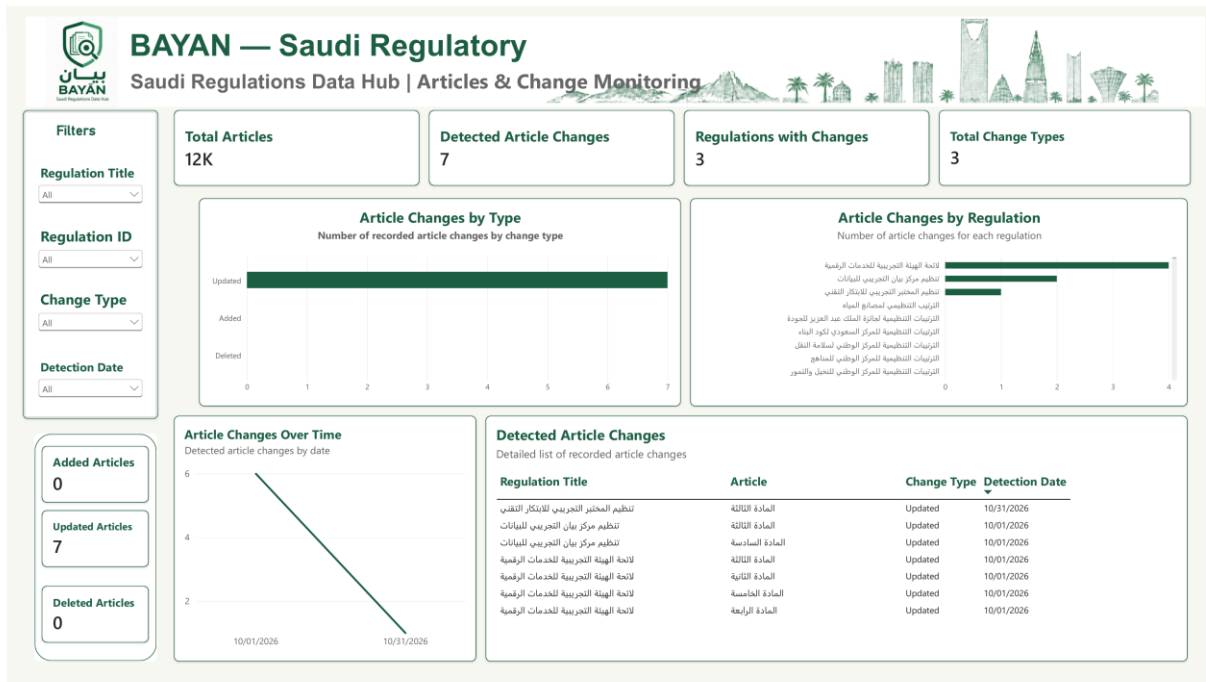


Figure 6: BAYAN Articles & Change Monitoring dashboard. Change events shown here include controlled validation/demo records.

The change-monitoring page reports approximately 12K articles, seven detected article changes, three regulations with changes, and three defined change types. In the current Gold implementation, the recorded fact events are Updated events. The displayed change examples were created as controlled test/demo data to validate versioning and change-flow behavior; they should not be interpreted as seven independently verified real-world regulatory amendments.

## 10. Development Process and Project Evolution

The proposal organized delivery into five phases: understand, design, build, test and evaluate, and deploy and present. Early work clarified the project idea, reviewed the official source, defined scope, and divided responsibilities. The design phase focused on the pipeline architecture, database schema, fields, workflow, and tools. The build phase implemented collection, cleaning, hashing, and change detection. Testing focused on data quality, version behavior, integration, and issue resolution. The final phase integrated the components, completed the dashboard, and prepared the final demonstration.

Several design decisions evolved during this process. The proposal described a broader search-and-comparison interface and line-by-line comparison as part of scope. The delivered MVP instead prioritized a reliable regulatory data foundation and Power BI analytics, while the dedicated frontend and detailed text diff were moved to the roadmap. This scope refinement reflects an engineering decision to complete the core ingestion, versioning, orchestration, and analytics layers before expanding the user application.

## 11. Limitations and Future Work

### 11.1 Current Limitations

- Current source coverage is focused on the Bureau of Experts rather than all Saudi regulatory authorities.
- Extraction depends on the current structure and availability of the source website.
- The submitted environment is a capstone implementation and is not presented as a fully production-hardened enterprise platform.
- Gold change-fact loading currently records Updated article events; Added and Deleted loading remains incomplete.
- The current analytical interface is Power BI; a public frontend portal and user accounts are not implemented.
- Detailed line-by-line visualization of added, removed, and modified legal text is not yet implemented.
- Real-time alerts, personal watchlists, and broader source integration remain future capabilities.

### 11.2 Future Work

The architecture is intended to grow beyond the current BOE-centered MVP. Future work includes integrating additional official Saudi regulatory sources, providing a dedicated frontend portal, adding advanced search and filtering, supporting user profiles and saved regulations, exposing APIs or data services, and introducing notification workflows for regulatory updates.

A high-value next step is detailed version comparison. Because the current model already preserves regulation versions and article versions, a future interface can compare two versions side by side and highlight text that was added, removed, or modified. This would move the user experience from answering “What is the regulation now?” toward answering “What changed between these versions?”

The proposal also identified OCR for scanned or image-based PDFs and intelligent summarization or regulatory-analysis features as later extensions. These are not part of the current system and would require separate design, evaluation, and governance before being introduced.

### 11.3 Expected Impact

The practical value of BAYAN is a structured and traceable regulatory data foundation. By separating raw evidence from normalized and analytical representations, the platform can support more consistent monitoring, easier historical analysis, and future services that depend on reliable regulatory data. The current design is deliberately extensible so the data layer can support richer applications without rebuilding the entire ingestion process.

## 12. Conclusion

BAYAN demonstrates how Saudi regulatory information can be engineered as a version-aware data product rather than handled only as separate documents. The final system collects official BOE content, preserves raw artifacts, structures regulatory entities in PostgreSQL, maintains version history, orchestrates recurring processing with Airflow, prepares analytics-ready Gold data, validates key relationships, and exposes results through Power BI. The project therefore delivers the core technical foundation required for regulatory monitoring while remaining clear about features that are not yet implemented.

The strongest outcome is the connection between source preservation, structured modeling, version tracking, and analytics. This creates a practical path for future expansion to additional official sources, detailed version comparison, alerts, APIs, and richer regulatory-intelligence services without discarding the existing data foundation.

## Acknowledgement

The team thanks the Saudi Digital Academy for the learning opportunity and program support, and WeCloudData for mentorship, technical guidance, and support throughout the capstone project. Project team: Norah Aldakhil, Yaqeen Alhalal, and Shouq Alotaibi.

## 13. References

- [1] BAYAN final project package: proposal, final presentation, source code, Airflow DAG, SQL files, Bronze snapshot, and Power BI dashboard assets, Sep. 2026.
- [2] Qanoniah, "Saudi law is easily accessible through Qanoniah," <https://qanoniah.com/en>, accessed Sep. 29, 2026.
- [3] Tashree, Saudi legal and regulatory document platform, <https://tashree.app/>, accessed Sep. 29, 2026.
- [4] Apache Airflow Documentation, "Cron & Time Intervals," <https://airflow.apache.org/docs/apache-airflow/stable/authoring-and-scheduling/cron.html>, accessed Sep. 29, 2026.
- [5] SeaweedFS, official project documentation and S3-compatible object storage, <https://github.com/seaweedfs/seaweedfs>, accessed Sep. 29, 2026.
- [6] PostgreSQL Documentation, <https://www.postgresql.org/docs/current/>, accessed Sep. 29, 2026.
- [7] Docker Documentation, "Docker Compose," <https://docs.docker.com/compose/>, accessed Sep. 29, 2026.
- [8] Microsoft Learn, "Power BI documentation," <https://learn.microsoft.com/en-us/power-bi/>, accessed Sep. 29, 2026.

## Appendix A - Selected Implementation Evidence

### A.1 Airflow schedule and dependency chain

```
RIYADH_TZ = pendulum.timezone("Asia/Riyadh")
@dag(
    dag_id="boe_pipeline",
    schedule="0 1 * * *",
    catchup=False,
    max_active_runs=1,
)
...
bronze >> silver >> gold
```

### A.2 Gold change types

```
INSERT INTO gold.dim_change_type (change_type_key, change_type)
VALUES
    (1, 'Added'),
    (2, 'Updated'),
    (3, 'Deleted');
```

### A.3 Current Gold change-event loading

```
INSERT INTO gold.fact_article_changes (... , change_type_key)
SELECT ..., 2
FROM silver.articles_versions av
JOIN silver.laws_versions lv ON av.law_version_id = lv.id
WHERE av.is_changed = true;
```

### A.4 Gold Analytical Structures

- gold.dim\_regulations
- gold.dim\_articles
- gold.dim\_date
- gold.dim\_change\_type
- gold.fact\_article\_changes

# Run Gold refresh

with open("refresh\_gold.sql", "r", encoding="utf-8") as file:

```
sql = file.read()
```

```
cursor.execute(sql)
```

```
connection.commit()
```

-- Build analytics-ready regulation dimension

```
INSERT INTO gold.dim_regulations (...)
```

```
SELECT ...
```

```
FROM silver.laws l
```

```
LEFT JOIN silver.laws_versions lv
```

```
ON l.id = lv.law_id;
```

-- Store detected article updates

```
INSERT INTO gold.fact_article_changes (...)
```

```
SELECT ...
```

```
FROM silver.articles_versions av
```

```
JOIN silver.laws_versions lv
```

```
ON av.law_version_id = lv.id
```

```
WHERE av.is_changed = true;
```